Build from source (CMake)

SWAT+ is built out of source: all intermediate and final files go into a build folder, so the source tree stays clean and several builds (different compilers, debug and release) can exist side by side.

The build is defined in `CMakeLists.txt` at the root of the repository. It compiles every `.f90` file in `src/` into one executable.

Requirements

- CMake 3.22 or newer
- a Fortran compiler: gfortran, Intel `ifx`, or Intel `ifort` (`ifort` is no longer included in Intel oneAPI from the 2025 release on)
- Git, which the build uses to find the version number. Configuration stops if Git is not installed; in a copy of the source that is not a Git clone (a downloaded archive), the version becomes `unknown`.
- `make` or Ninja

Configure

From the `swatplus` folder:

```
cmake -B build -D CMAKE_BUILD_TYPE=Release
```

This writes the build system into `build/` without compiling anything. Give a build type: `Release` for an optimized executable, `Debug` for one with run-time checks and debugger symbols. Without a build type neither the optimization nor the debug flags are added.

To choose the compiler:

```
cmake -B build -D CMAKE_BUILD_TYPE=Release -D CMAKE_Fortran_COMPILER=gfortran
cmake -B build -D CMAKE_BUILD_TYPE=Release -D CMAKE_Fortran_COMPILER=ifx
```

On Windows, configure from the Intel oneAPI command prompt to use the Intel compilers, or use Ninja with gfortran:

```
cmake -B build -D CMAKE_GENERATOR_TOOLSET=fortran=ifx
cmake -B build -G "Ninja" -D CMAKE_Fortran_COMPILER=gfortran -D CMAKE_BUILD_TYPE=Release
```

`CMakePresets.json` also defines debug and release presets for gfortran (Linux, Windows, macOS), `ifx` and `ifort`; list them with `cmake --list-presets`.

Compile

```
cmake --build build -j 8
```

The executable is written to `build/` as

```
swatplus-<version>-<compiler>-<system>_<arch>[-Rel|-Dbg][.exe]
```

for example `swatplus-62.0.1-gnu-lin_x86_64-Rel`. The compiler part is `gnu`, `ifx` or `ifo` (Intel Fortran Classic), and the version comes from the Git tag (see Tagging and releases).

To start again from scratch, delete the build folder (`rm -rf build`, or `rd /s /q build` in Windows `cmd`). To remove only the compiled files and keep the configuration:

```
cmake --build build --target clean
```

Compiler options

Compiler	Always	Release	Debug
gfortran (Linux, macOS)	free form, no line length limit, implicit none, local variables set to zero, strict rounding	`-O`	`-Wall -fcheck=all -ffpe-trap=invalid,zero,overflow,unde -fsignaling-nans`
gfortran (Windows)	as above, plus `-fcheck=all` and the floating-point traps	`-O`	`-Wall`
`ifx`	free form, `-fpe0`, traceback	`-O`	`-warn all -O0 -check bounds`
`ifort`	free form, `-fpe0`, traceback	`-O`	`-warn all`

From 62.0.1 the gfortran run-time checks and floating-point traps are used only in Debug builds on Linux and macOS. Earlier releases also applied them to Release builds, and those executables could stop with an illegal instruction error on macOS when a system math routine raised a harmless floating-point flag.

Linking

- Linux, gfortran: linked statically, except on Arch, Arch-based and Fedora systems, which do not ship the static `libgfortran`; there the executable needs the gfortran run-time libraries at run time (`ldd <executable>` lists them).
- macOS, gfortran: the gfortran, quadmath and gcc run-time libraries are linked statically, so the executable runs on Macs without Homebrew's gcc installed. A fully static macOS executable is not possible.
- Intel compilers and Windows gfortran: linked statically.

Install

`cmake --install build` copies the executable to the root of the source folder, whatever `--prefix` is given. To put it somewhere else, copy the file from `build/` yourself.

Sources

- CMakeLists.txt: version from `git describe --tags` (lines 20 to 34), compiler flags (`fdialect`, `fdebug`, `frelease`) and static linking (lines 36 to 100), executable name (`SWATPLUS_EXE`), `install(TARGETS ${SWATPLUS_EXE} DESTINATION RUNTIME DESTINATION ${PROJECT_SOURCE_DIR})`.
- CMakePresets.json and doc/Building.md.